



A Project Development Documentation

On

PRT-18

REAL-TIME UNIVERSITY BUS TRACKING SYSTEM

Living Document — Updated Continuously as Development Progresses

Master of Computer Application (MCA)

Submitted by

Team Size: 2 Members

Team Member 1 :- Birendra Prasad - 25PG00018)

Team Member 2:- (Meghana R - 25PG00174)

Under the Guidance of

Sarath Babu Sribhaskyam Sir

Table of Contents

Document Purpose	5
1. Project Overview	6
2. Project Objectives	6
3. User Roles	7
4. What We Have Completed	8
5. Backend Development	11
6. Database	12
7. System Architecture	13
8. Current Data Flow	16
9. Login Flow	17
10. Admin Flow	19
11. Student Flow	19
12. Driver Flow	21
13. Complete Real-Time System Flow (Target / Planned)	21
14. API Documentation	25
15. Testing Completed	27
16. Current Status	28
17. Development Roadmap	29
18. What We Should Do Next	31

List of Figures

Figure 1: Entity-Relationship diagram for university_bus_tracking, with implemented vs. planned tables clearly marked.

Figure 2: Overall system architecture — the implemented request/response path.

Figure 3: Planned real-time location pipeline — target architecture, not yet implemented.

Figure 4: Planned analytics pipeline via Apache Superset.

Figure 5: Data flow diagram for the currently implemented system.

Figure 6: Login flow — authentication and authorization.

Figure 7: Admin flow: authentication, authorization, and the management APIs it unlocks.

Figure 8: Student flow, with each section marked by its real implementation status.

Figure 9: Planned driver flow, from login through to GPS reaching the live map.

Figure 10: GPS data flow — target design, not yet implemented.

Figure 11: Kafka streaming flow — target design, not yet implemented.

Figure 12: ETA calculation flow — target design, not yet implemented.

Figure 13: Complete target system flow — end-to-end, PLANNED architecture.

Figure 14: Twelve-phase development roadmap, current progress marked.

List of Tables

Table 1: Project objectives and current status.

Table 2: Role responsibilities and current implementation status.

Table 3: Repository and project-structure work completed.

Table 4: Environment, backend, and database configuration work completed.

Table 5: Data models and management API work completed.

Table 6: Authentication, authorization, and API documentation work completed.

Table 7: Frontend work completed.

Table 8: Purpose of each backend folder.

Table 9: Database tables, purpose, and implementation status.

Table 10: Student dashboard sections: UI status vs. backend status.

Table 11: Implemented API endpoints, with purpose, authentication requirement, and expected result.

Table 12: Manual tests performed to date, all executed against the implemented backend and frontend.

Table 13: Feature-by-feature project status as of this revision of the document.

Table 14: Development roadmap phases and current status.

Document Purpose

This document is a concise, accurate, and continuously updated development record for PRJ-18, the Real-Time University Bus Tracking System. It is not a final academic thesis. Its purpose is to give any evaluator, mentor, or new team member three clear answers: what has actually been developed and tested so far; how the complete system is designed and intended to work end-to-end; and what remains to be built, in what order.

Throughout this document, every component, API, table, and diagram element is explicitly tagged with one of three statuses:

Status Tag	Meaning
COMPLETED / IMPLEMENTED	Built, connected, and verified to work as described.
IN PROGRESS	Partially built — e.g. UI exists but backend integration is incomplete.
PLANNED	Designed on paper / in the schema, but not yet implemented or working end-to-end.

No component described as PLANNED in this document — including live GPS tracking, Kafka streaming, ETA calculation, or Superset analytics — is currently functional. These are target-architecture items only, included to document the design intent.

1. Project Overview

PRJ-18, The Real-Time University Bus Tracking System, is a web-based platform being built to help students, faculty, drivers, and administrators manage and monitor university bus transport from a single system. In its target form, the system will let a driver's phone report the live location of a bus, and let students and faculty see that bus moving on a map along with an estimated time of arrival at their stop.

The problem it solves is a familiar one on university campuses: students and faculty currently have no reliable way to know where a bus is, whether it is delayed, or when it will actually arrive at their stop. This leads to wasted waiting time, missed buses, and no central visibility for transport administrators into how the fleet is being used. A real-time tracking system removes this uncertainty by making bus location, route, and schedule information available digitally and, eventually, live.

Real-time tracking is especially useful in a university context because bus schedules are shared across a large number of routes and stops, ridership is high during specific class-change windows, and the administration needs data (not guesswork) to plan routes, assign drivers, and respond to delays.

When complete, the final system will provide: role-based dashboards for students, faculty, drivers, and administrators; a searchable directory of buses, routes, and stops; a live map showing each bus's current position; an estimated time of arrival (ETA) for each stop; and an analytics dashboard for administrators built on the data the system collects. As of this document's writing, the management, authentication, and dashboard-UI portions of this vision are implemented; the live-tracking, ETA, and analytics portions are designed but not yet functional — this distinction is maintained throughout the document.

2. Project Objectives

The project is being developed against the following objectives. Each objective is tagged with its current status so the reader does not need to infer progress from prose.

Objective	Description	Status
Manage university buses	Create, list, and update bus records (registration number, capacity, assigned driver).	COMPLETED

Objective	Description	Status
Manage routes	Create and list routes that buses operate on.	COMPLETED
Manage stops	Create stops and attach them to a route in sequence.	COMPLETED
Manage drivers	Register drivers and assign them to buses.	COMPLETED
Authenticate users	Secure login using hashed passwords and JWT tokens.	COMPLETED
Provide role-based access	Restrict admin-only and driver-only endpoints by role.	COMPLETED
Let students/faculty view bus information	Dashboards to browse buses, routes, and stops.	IN PROGRESS
Provide live bus tracking	Show a bus's real-time position on a map.	PLANNED
Calculate ETA	Estimate arrival time at each stop from live position and speed.	PLANNED
Stream GPS data using Kafka	Move location updates from driver to database via a Kafka pipeline.	PLANNED
Provide transport analytics using Superset	Route/trip/ETA metrics for administrators.	PLANNED

Table 1: Project objectives and current status.

Real-time tracking, ETA calculation, Kafka streaming, and Superset analytics are explicitly PLANNED objectives. No claim is made anywhere in this document that they are currently functional.

3. User Roles

The system is designed around four conceptual user roles: Student, Driver, Faculty, and Admin. This four-role model is a design decision made by our team for PRJ-18 — it is not dictated by any external specification — chosen because it maps cleanly onto how a university transport system is actually used: two groups of riders (students and faculty) with near-identical needs, one operational role (driver) that generates data, and one administrative role that manages the whole system.

Role	Responsibilities	Backend Status	Frontend Status
Student	Login, Dashboard, My Routes, Live Tracking, Bus Details, Timetable, Notifications, Profile	Partially implemented (auth + core data APIs)	Dashboard UI built

Role	Responsibilities	Backend Status	Frontend Status
Driver	Login, Assigned Bus, Assigned Route, Start Trip, GPS Sharing, Trip History	Data model only; GPS/trip APIs planned	Not yet built
Faculty	Login, Routes, Live Tracking, Bus Details, Timetable, Notifications, Profile	Shares Student-level APIs	Not yet built
Admin	User management, Driver management, Bus management, Route management, Stop management, Trip management, Live monitoring, Reports, Analytics	Management APIs implemented; monitoring/reports planned	Not yet built (Swagger used for now)

Table 2: Role responsibilities and current implementation status.

4. What We Have Completed

This chapter lists only the work that has actually been built and verified to run. Items are grouped into five categories for readability. Each table states what was done, the technology used, and the current, observable result.

4.1 Repository, Project Structure & Version Control

A clean repository structure was established first so that backend, frontend, database, and future Kafka/Superset work stay isolated and easy to navigate as the team grows the codebase.

Item	What Was Done	Technology	Current Result
GitHub repository	Central repository created and shared between both team members.	Git + GitHub	Repo live at github.com/BirendraPrasad/University-Bus-Tracking
Project folder structure	Top-level folders separated by concern.	Filesystem convention	backend/, frontend/, database/, kafka/, superset/ exist
Backend folder	Dedicated FastAPI application folder.	FastAPI project layout	Contains app/ with models, schemas, routers, services
Frontend folder	Static HTML/CSS/JS app folder.	Plain HTML/CSS/JS	Contains landing, login, and dashboard pages
Database folder	Holds schema/SQL assets for PostgreSQL.	SQL	Schema scripts kept alongside code
Kafka folder (placeholder)	Reserved location for future producer/consumer scripts.	Apache Kafka (planned)	Folder exists; empty of working code
Superset folder (placeholder)	Reserved location for future dashboard config/export.	Apache Superset (planned)	Folder exists; empty of working code
Git commit history	Regular commits pushed as features were completed.	Git + GitHub	Incremental commit history visible on GitHub

Table 3: Repository and project-structure work completed.

4.2 Environment, Backend & Database Configuration

Before any feature could be built, the local development environment, the FastAPI server, and the PostgreSQL database had to be installed, configured, and connected to each other.

Item	What Was Done	Technology	Current Result
Python virtual environment	Isolated environment created for backend dependencies.	venv	Reproducible local backend setup
FastAPI installed/configured	Core web framework installed and app entry point created.	FastAPI	App boots and serves requests
Uvicorn configured	ASGI server configured to run the FastAPI app.	Uvicorn	Server runs locally with auto-reload
PostgreSQL installed/configured	Database server installed locally.	PostgreSQL	Server running and reachable
PostgreSQL database created	Dedicated database created for the project.	PostgreSQL	university_bus_tracking database exists
PostgreSQL user/role configured	Application-specific DB user with scoped privileges.	PostgreSQL roles	Backend connects using a dedicated role, not superuser
Database schema created	Core tables created (see Chapter 6).	SQL / SQLAlchemy models	Schema present in the database
FastAPI connected with PostgreSQL	ORM/DB session wired into the FastAPI app.	SQLAlchemy	Endpoints read/write real DB rows
.env configuration	Secrets and connection strings externalised from code.	python-dotenv	DB URL, JWT secret loaded from .env
.gitignore configured	Prevents venv, .env, and cache files from being committed.	Git	Repository stays clean of local artefacts

Table 4: Environment, backend, and database configuration work completed.

4.3 Data Models & Management APIs

With the database connected, core entities were modelled and exposed through REST endpoints so that buses, routes, stops, and people involved in transport can be created and queried.

Item	What Was Done	Technology	Current Result
User model/schema	Base user table + Pydantic schemas for request/response validation.	SQLAlchemy + Pydantic	Shared identity for all four roles
Student management	Create/list student records linked to a user.	FastAPI router + service	Students can be registered via API
Driver management	Create/list driver records linked to a user.	FastAPI router + service	Drivers can be registered via API
Faculty management	Create/list faculty records linked to a user.	FastAPI router + service	Faculty can be registered via API
Bus management	Create/list buses.	FastAPI router + service	Buses can be created and fetched

Item	What Was Done	Technology	Current Result
Route management	Create/list routes.	FastAPI router + service	Routes can be created and fetched
Stop management	Create stops under a route; list stops per route.	FastAPI router + service	Stops can be created and fetched per route
Driver-bus assignment	Assign a driver to a specific bus.	FastAPI PUT endpoint	Bus record updated with assigned driver

Table 5: Data models and management API work completed.

4.4 Authentication, Authorization & API Documentation

Security was treated as a first-class concern rather than an afterthought: passwords are never stored in plain text, sessions are stateless via JWT, and role checks are enforced on the server, not just hidden in the UI.

Item	What Was Done	Technology	Current Result
Password hashing	Passwords hashed before storage; never stored as plain text.	bcrypt	Verified: DB never contains raw passwords
JWT authentication	Stateless token issued on login and required on protected routes.	PyJWT / python-Jose	Token issued and validated per request
Role-based authorization	Endpoints check the caller's role before executing.	FastAPI dependencies	Non-admins receive 403 on admin routes
Login API	Verifies credentials and issues a JWT.	FastAPI + bcrypt	POST /auth/login returns a working token
Registration API	Creates a new user account.	FastAPI + Pydantic	POST /users/ creates a user
Public Driver/Admin registration blocked	Self-registration restricted to Student role at the API layer.	Server-side role check	Attempting to self-register as Driver/Admin is rejected
Admin-protected APIs	Sensitive endpoints require an authenticated Admin token.	FastAPI dependencies	Verified via GET /users/admin-test
Swagger/Open API	Auto-generated interactive API docs.	FastAPI built-in	Available and used for manual testing

Table 6: Authentication, authorization, and API documentation work completed.

4.5 Frontend

The frontend work completed so far focuses on the public-facing pages and the student experience, since students are the largest user group. Faculty, driver, and admin interfaces are planned next (see Chapter 17).

Item	What Was Done	Technology	Current Result
Landing page	Public entry page introducing the system.	HTML/CSS/JS	Loads and links to login
Login page	Form for email + password, calls the login API.	HTML/CSS/JS + fetch	Successful login redirects to dashboard
Student dashboard	Main authenticated view for students.	HTML/CSS/JS	Loads after login with a token
Single-page navigation	Dashboard sections switch without full page reloads.	Vanilla JS DOM manipulation	Smoother in-dashboard navigation
Live tracking map UI	Map container and layout built into the dashboard.	HTML/CSS	Map renders; no live bus data yet
Leaflet + OpenStreetMap	Interactive map library integrated and initialised.	Leaflet.js + OSM tiles	Map pans/zooms with OSM tiles

Table 7: Frontend work completed.

The live tracking map renders correctly and is ready to receive data, but it is not yet connected to any real bus position — see Chapter 12 and Chapter 16 for status.

5. Backend Development

The backend follows a layered folder structure so that request handling, validation, business logic, and data access remain separate and easy to maintain as the codebase grows:

```

backend/ └─ app/      └─ models/      (SQLAlchemy ORM table definitions)
    └─ schemas/      (Pydantic request/response validation)      └─ routers/
    (FastAPI route/endpoint definitions)      └─ services/      (business logic,
    called by routers)      └─ database/      (DB session/engine setup)      └─
main.py      (FastAPI app entry point)

```

5.1 Purpose of Each Folder

Folder	Purpose
models/	Defines database tables as SQLAlchemy ORM classes (Users, Students, Drivers, Buses, Routes, Stops, etc.).
schemas/	Defines Pydantic models used to validate incoming request bodies and shape outgoing JSON responses.
routers/	Declares the actual API endpoints (paths, HTTP methods) and delegates to services.
services/	Contains the business logic — e.g. checking role, hashing a password, assigning a driver — kept separate from the HTTP layer.
database/	Sets up the SQLAlchemy engine, session factory, and dependency used to obtain a DB session per request.
main.py	Creates the FastAPI app instance, registers routers, and starts the application.

Table 8: Purpose of each backend folder.

5.2 How a Request Moves Through the Backend

Every implemented API call follows the same layered path, which keeps the codebase predictable and testable:

1. A request reaches FastAPI's ASGI server (Uvicorn).
2. FastAPI matches the URL and method to a Router.
3. The Router passes the request body through a Schema for validation.
4. Valid data is handed to a Service, which applies business rules (e.g. role checks, password hashing).
5. The Service reads or writes through the Database layer to PostgreSQL.
6. The result flows back up: Database → Service → Router → JSON Response to the client.

This exact flow is illustrated as a diagram in Chapter 8 (Current Data Flow).

6. Database

The project uses a single PostgreSQL database named `university_bus_tracking`. The tables below fall into two groups: core tables that are created and actively used by implemented APIs, and tables that exist in the design for future real-time functionality (trip tracking, live GPS, subscriptions, schedules, notifications) which are not yet driven by working application logic.

Table	Purpose	Status
users	Base identity table (email, hashed password, role) shared by all four roles.	COMPLETED
students	Student-specific profile data, linked to a users row.	COMPLETED
drivers	Driver-specific profile data, linked to a users row.	COMPLETED
faculty	Faculty-specific profile data, linked to a users row.	COMPLETED
buses	Bus fleet records (registration, capacity, assigned driver).	COMPLETED
routes	Named routes that buses operate on.	COMPLETED
stops	Stops belonging to a route, in sequence.	COMPLETED
trips	A single run of a bus along a route (planned unit of GPS/ETA tracking).	PLANNED
bus_locations	Time-stamped GPS points per bus, written by the future Kafka consumer.	PLANNED
subscriptions	Links a student to the route(s) they follow, for personalised tracking/notifications.	PLANNED
schedules	Planned/scheduled timing information per route.	PLANNED
notifications	Delay/arrival alerts to be pushed to users.	PLANNED

Table 9: Database tables, purpose, and implementation status.

Tables marked **PLANNED** are designed as part of the target schema so that future work (Chapters 12–13) has a clear destination, but they are not yet populated or read by working application logic.

6.1 Entity-Relationship Diagram

The diagram below shows how the core (implemented) tables relate to each other, and how the planned real-time tables attach to that core design.

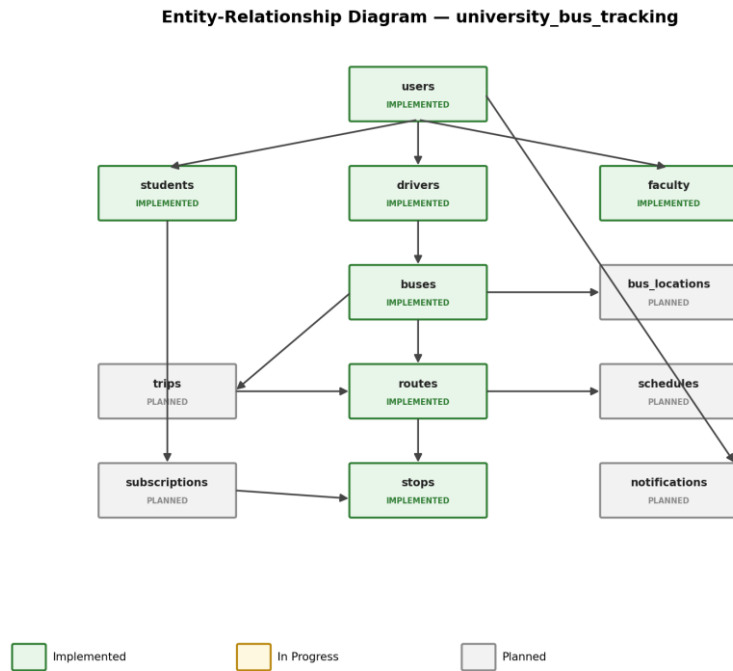


Figure 1: Entity-Relationship diagram for university_bus_tracking, with implemented vs. planned tables clearly marked.

7. System Architecture

The system architecture is presented in three parts: the implemented request/response path used today, the planned real-time GPS pipeline, and the planned analytics pipeline. These are kept visually and physically separate so that implemented and planned work are never confused.

7.1 Implemented Architecture

All currently working features — authentication, and management of students, drivers, faculty, buses, routes, and stops — flow through this path.

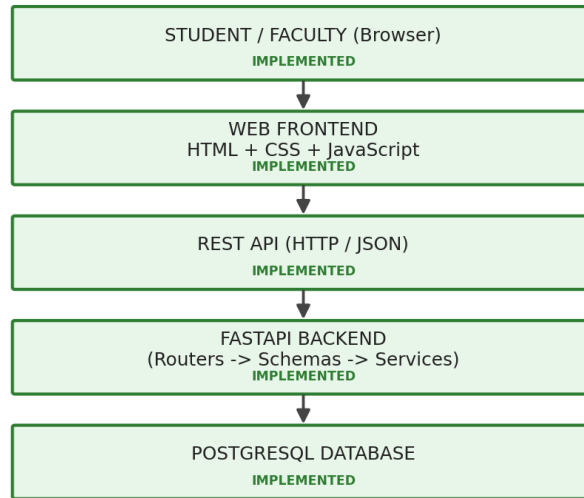
Overall System Architecture -- Implemented Path

Figure 2: Overall system architecture — the implemented request/response path.

7.2 Planned Real-Time Location Pipeline

This pipeline describes how a driver's live GPS position is intended to reach a student's map, once Phases 5–8 of the roadmap (Chapter 17) are complete. None of the stages below are functional yet.

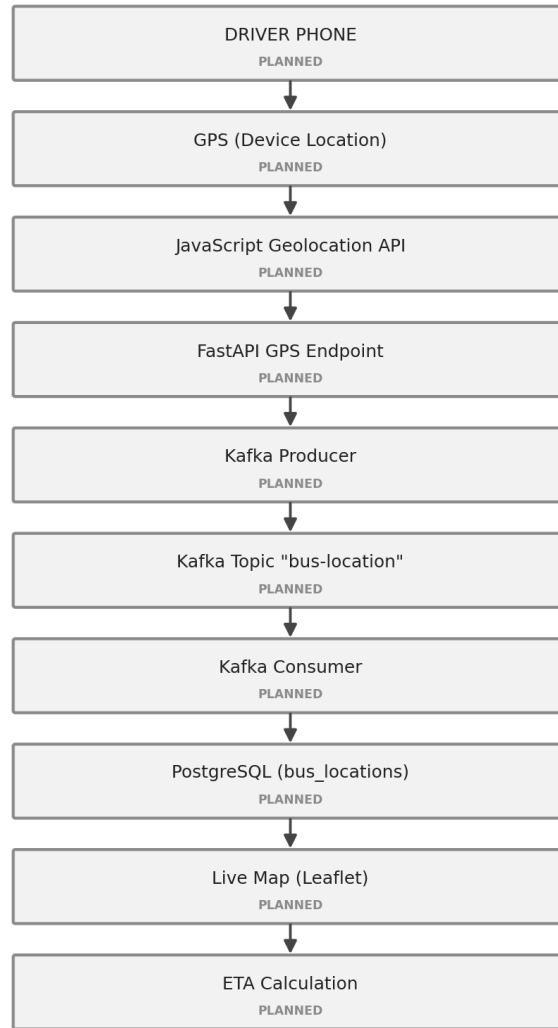
Planned Real-Time Location Pipeline (Target Architecture)

Figure 3: Planned real-time location pipeline — target architecture, not yet implemented.

7.3 Planned Analytics Pipeline

PostgreSQL already holds the operational data Superset would need; the Superset connection and dashboards themselves are planned for a later phase.

Planned Analytics Pipeline

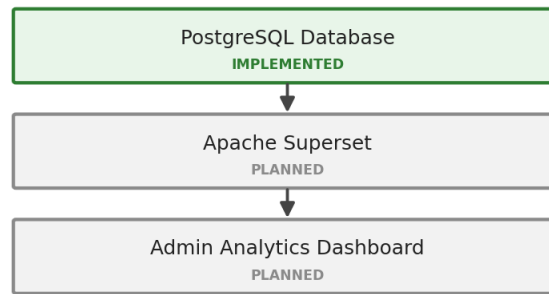


Figure 4: Planned analytics pipeline via Apache Superset.

8. Current Data Flow

The data flow diagram below documents exactly how a request is currently processed end-to-end for any implemented feature (for example, an admin creating a bus, or a student logging in). Every stage shown here is implemented and has been exercised through Swagger and the frontend.

Current Implemented Data Flow

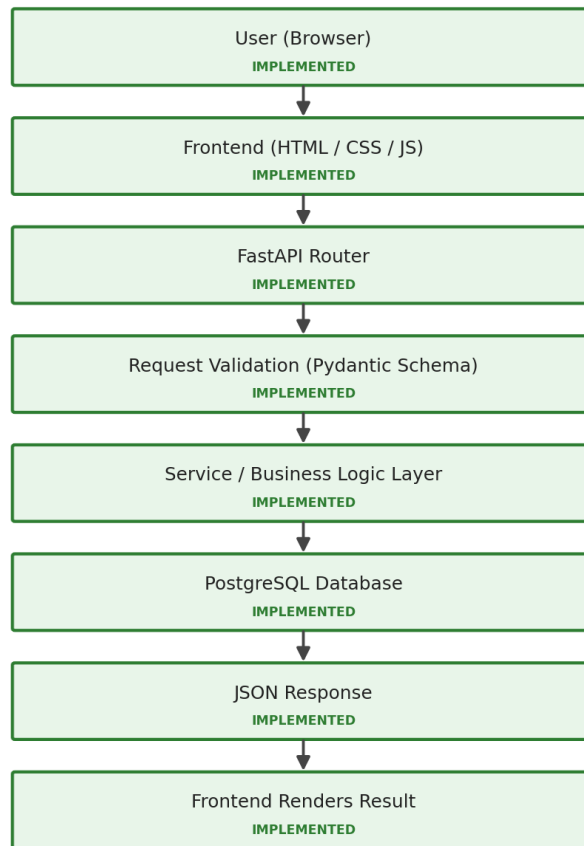
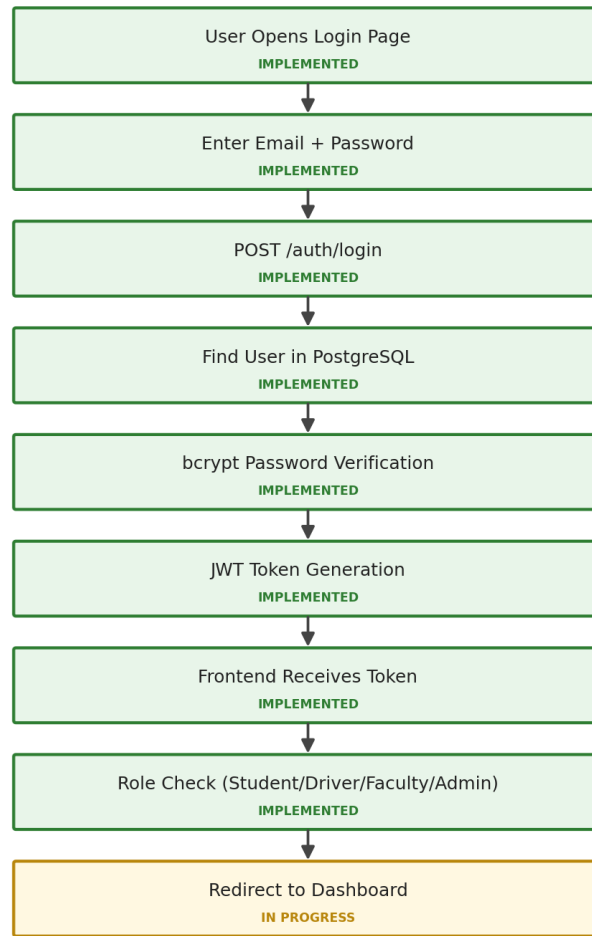


Figure 5: Data flow diagram for the currently implemented system.

In words: the user interacts with the Frontend, which sends an HTTP request to a FastAPI Router. The Router validates the incoming payload against a Schema. Valid requests reach the Service layer, which applies business logic and talks to PostgreSQL. The result is serialised back into JSON and returned to the Frontend, which updates the UI.

9. Login Flow

Authentication (proving who the user is) and authorization (deciding what that user may do) are implemented as two distinct steps, which is a deliberate security design choice.

Login Flow (Authentication + Authorization)*Figure 6: Login flow — authentication and authorization.*

9.1 Authentication

The user submits an email and password to POST /auth/login. The backend looks up the user in PostgreSQL, and verifies the submitted password against the bcrypt hash stored for that user. If it matches, a JWT is generated and returned to the frontend. This proves identity only — it does not yet decide what the user can access.

9.2 Authorization

Once the frontend holds a JWT, every subsequent request includes it. Protected endpoints decode the token, read the embedded role claim, and only proceed if that role is permitted for the endpoint (for example, admin-only endpoints reject Student or Driver tokens with a 403 response). This role check happens on the backend, not just by hiding buttons in the UI, which is what makes it a genuine authorization control rather than a cosmetic one.

10. Admin Flow

An admin logs in like any other user, but the resulting JWT carries the Admin role. Every admin API additionally passes through an authorization check before reaching the underlying management logic.

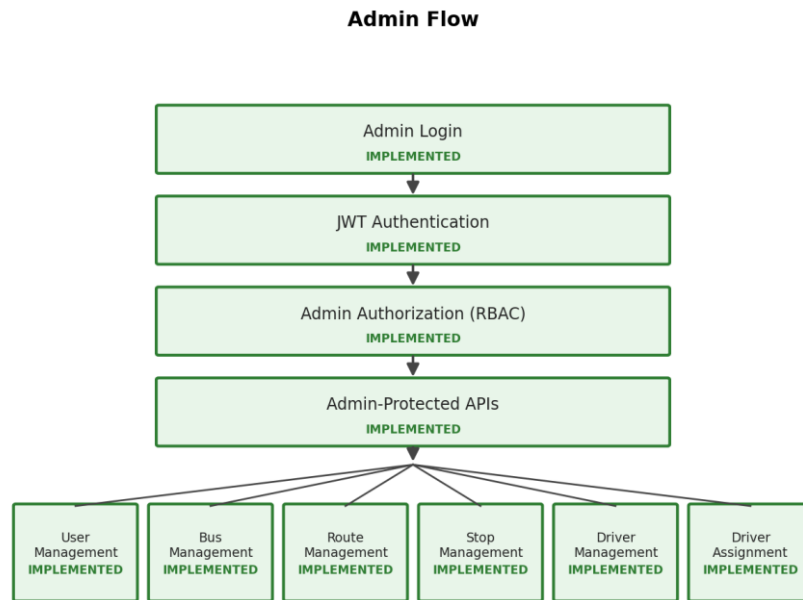


Figure 7: Admin flow: authentication, authorization, and the management APIs it unlocks.

All six management capabilities shown above (User, Bus, Route, Stop, Driver management, and Driver Assignment) are implemented as working backend APIs today. A dedicated Admin dashboard UI to drive them visually is planned (see Chapter 17); until then, these APIs are exercised through Swagger/OpenAPI.

11. Student Flow

The student flow reflects the actual state of the student-facing frontend: the dashboard shell, navigation, and map UI exist, but most sections are not yet backed by working data APIs. This chapter is explicit about which parts are UI-only.

Student Flow

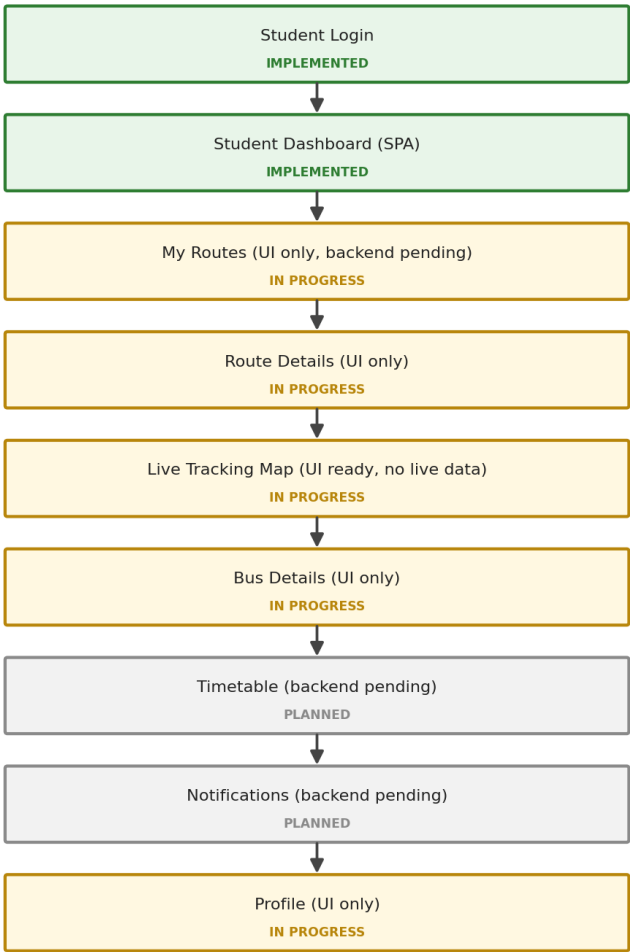


Figure 8: Student flow, with each section marked by its real implementation status.

Section	What Exists Today	Backend Status
Login	Full login form, calls the real login API.	COMPLETED
Dashboard	Single-page dashboard shell with navigation.	COMPLETED
My Routes	UI section built.	IN PROGRESS
Route Details	UI section built.	IN PROGRESS
Live Tracking	Map renders with Leaflet + OSM, no live bus data.	IN PROGRESS
Bus Details	UI section built.	IN PROGRESS
Timetable	Not yet built.	PLANNED
Notifications	Not yet built.	PLANNED
Profile	UI section built.	IN PROGRESS

Table 10: Student dashboard sections: UI status vs. backend status.

12. Driver Flow

The driver flow is the planned entry point into the real-time pipeline. Today, a driver exists only as a data record (created and assignable to a bus by an admin); the driver-facing app, trip lifecycle, and GPS sharing described below are all planned work.

Driver Flow (Planned)

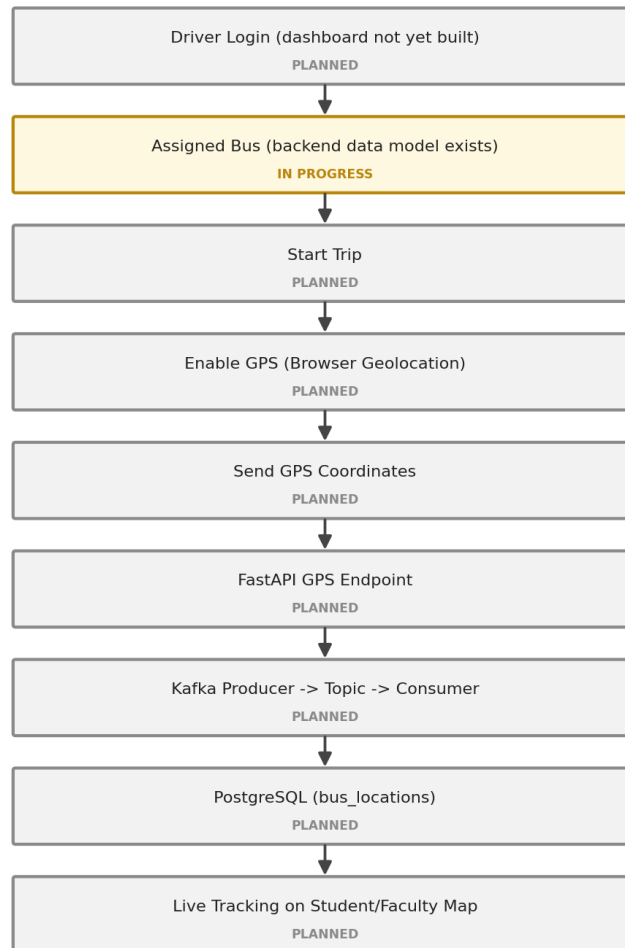


Figure 9: Planned driver flow, from login through to GPS reaching the live map.

GPS capture, the FastAPI GPS endpoint, and the Kafka stages in this flow are PLANNED. Only the underlying driver and bus-assignment data model is implemented today.

13. Complete Real-Time System Flow (Target / Planned)

This chapter documents, in detail, how the finished system is intended to work end-to-end. It is the single most important planning reference in this document and is written to be unambiguous: every stage described here is PLANNED. None of it is implemented yet. It exists to guide the work in Phases 5–11 of the roadmap (Chapter 17).

13.1 GPS Data Flow

The driver's phone will capture location through the browser's Geolocation API, send it to a dedicated FastAPI endpoint, and that endpoint will publish it onto a Kafka topic rather than writing directly to PostgreSQL — this decouples GPS ingestion speed from database write speed and allows multiple consumers later (e.g. live map, analytics).

GPS Data Flow (Planned)

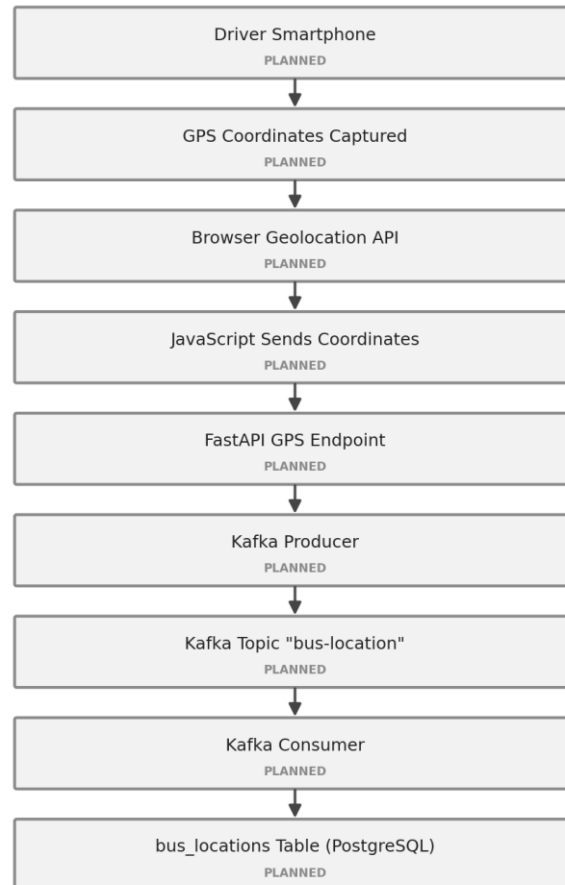


Figure 10: GPS data flow — target design, not yet implemented.

13.2 Kafka Streaming Flow

A Kafka Producer embedded in the GPS endpoint will publish each location update as a message on a topic named "bus-location". A Kafka Consumer will subscribe to that topic and persist each update into the bus_locations table. This is the core of the real-time pipeline and is entirely unimplemented at present — no producer, topic, or consumer exists yet.

Kafka Streaming Flow (Planned)

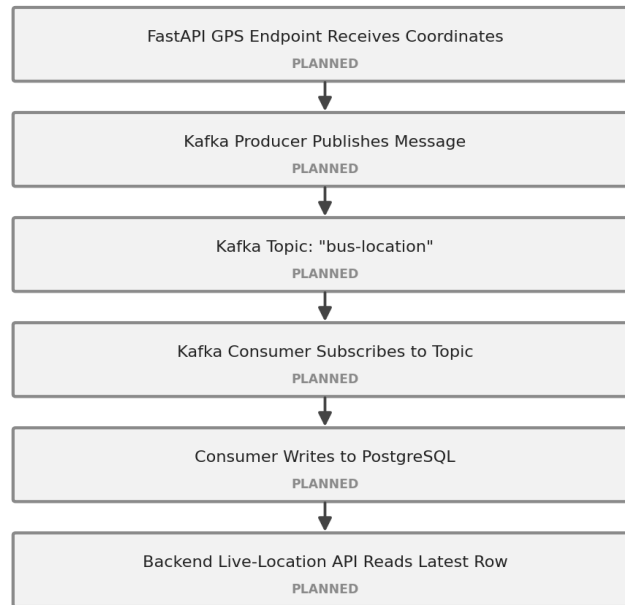


Figure 11: Kafka streaming flow — target design, not yet implemented.

13.3 ETA Calculation Flow

Once live positions are available, ETA will be derived (not looked up) from the bus's current GPS point, the next stop on its route, the straight-line/haversine distance between them, and a recent-speed estimate from the last few GPS points. This calculation, and the API that would expose it, are both planned and not yet built.

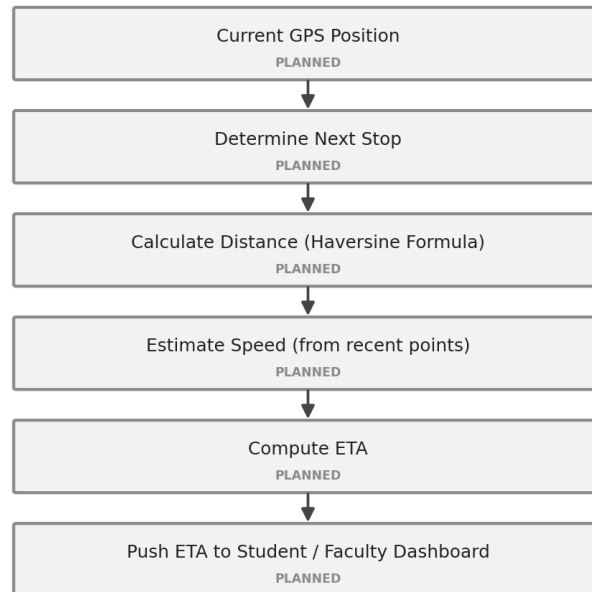
ETA Calculation Flow (Planned)

Figure 12: ETA calculation flow — target design, not yet implemented.

13.4 Complete Target Flow (End-to-End)

The diagram below combines the previous three flows into a single end-to-end picture, from the driver's phone to a moving marker on a student's or faculty member's map.

Complete Target System Flow (End-to-End, PLANNED)

Figure 13: Complete target system flow — end-to-end, PLANNED architecture.

13.5 Analytics Path

In parallel with live tracking, the same PostgreSQL data (trips, bus_locations, schedules) is intended to feed Apache Superset, giving administrators dashboards on metrics such as on-time trip starts, route usage, and ETA accuracy. This is planned for Phase 11 of the roadmap and has not been started.

14. API Documentation

This chapter documents only the APIs that actually exist and respond today, as exercised through Swagger/OpenAPI. Request field names are intentionally omitted where they are not already fixed in the implementation, to avoid documenting anything speculative.

Endpoint	Method	Purpose	Auth Required	Expected Result
GET /	GET	Basic health/welcome check for the API.	No	200 OK with a welcome message
GET /db-test	GET	Confirms the backend can reach PostgreSQL.	No	200 OK confirming DB connectivity
POST /auth/login	POST	Authenticates a user and issues a JWT.	No	200 OK with access token, or 401 on bad credentials
GET /users/	GET	Lists users.	Yes	200 OK with a list of user records
POST /users/	POST	Registers a new user (Student self-registration; Driver/Admin blocked).	No (public, restricted to Student role)	201 Created, or 403 if a non-Student role is attempted
GET /users/profile	GET	Returns the authenticated user's own profile.	Yes	200 OK with the caller's profile
GET /users/admin-test	GET	Verifies admin-only authorization is enforced.	Yes (Admin)	200 OK for Admin, 403 for other roles
POST /users/students	POST	Creates a student record.	Yes (Admin)	201 Created with the new student record
POST /users/drivers	POST	Creates a driver record.	Yes (Admin)	201 Created with the new driver record
GET /users/drivers	GET	Lists drivers.	Yes (Admin)	200 OK with a list of drivers
POST /users/faculty	POST	Creates a faculty record.	Yes (Admin)	201 Created with the new faculty record
POST /users/buses	POST	Creates a bus record.	Yes (Admin)	201 Created with the new bus record
GET /users/buses	GET	Lists buses.	Yes	200 OK with a list of buses
POST /users/routes	POST	Creates a route.	Yes (Admin)	201 Created with the new route record
GET /users/routes	GET	Lists routes.	Yes	200 OK with a list of routes
POST /users/stops	POST	Creates a stop under a route.	Yes (Admin)	201 Created with the new stop record
GET /users/routes/{route_id}/stops	GET	Lists stops belonging to a specific route.	Yes	200 OK with the route's stops in sequence

Endpoint	Method	Purpose	Auth Required	Expected Result
PUT /users/buses/{bus_id}/driver	PUT	Assigns a driver to a bus.	Yes (Admin)	200 OK with the updated bus record

Table 11: Implemented API endpoints, with purpose, authentication requirement, and expected result.

No GPS-ingestion, Kafka-related, ETA, subscription, notification, or Superset endpoint exists yet. These are listed only as planned work in Chapters 13, 17, and 18.

15. Testing Completed

Testing so far has been manual, performed through Swagger/OpenAPI and the browser, covering every implemented feature at least once. Automated test suites are not yet part of the project (see Chapter 18).

Test ID	Test	Expected Result	Actual Result	Status
T01	FastAPI startup	Server starts without errors	Server started successfully	PASS
T02	Root API (GET /)	Returns 200 with welcome message	200 OK returned	PASS
T03	Database connection (GET /db-test)	Confirms PostgreSQL connectivity	Connection confirmed	PASS
T04	Swagger UI	Interactive docs load at /docs	Docs loaded and usable	PASS
T05	Registration	New user created in database	User record created	PASS
T06	Login	Valid credentials return a JWT	Token returned	PASS
T07	Password verification	Wrong password is rejected	401 returned for bad password	PASS
T08	JWT validation	Protected route accepts a valid token	Access granted with valid token	PASS
T09	Admin authorization	Non-admin blocked from admin route	403 returned for non-admin	PASS
T10	Student creation	Admin can create a student	Student record created	PASS
T11	Driver creation	Admin can create a driver	Driver record created	PASS
T12	Faculty creation	Admin can create faculty	Faculty record created	PASS
T13	Bus creation	Admin can create a bus	Bus record created	PASS
T14	Bus retrieval	GET /users/buses returns bus list	Bus list returned	PASS
T15	Route creation	Admin can create a route	Route record created	PASS

Test ID	Test	Expected Result	Actual Result	Status
T16	Route retrieval	GET /users/routes returns route list	Route list returned	PASS
T17	Stop creation	Admin can create a stop under a route	Stop record created	PASS
T18	Stop retrieval	Stops returned for a given route, in order	Stops returned correctly	PASS
T19	Driver assignment	Driver linked to a bus via PUT endpoint	Bus record updated with driver	PASS
T20	Frontend login	Login page authenticates and redirects	Redirect to dashboard on success	PASS
T21	Student dashboard navigation	SPA sections switch without reload	Navigation works as expected	PASS

Table 12: Manual tests performed to date, all executed against the implemented backend and frontend.

15.1 Pending Tests

The following cannot be tested yet because the underlying functionality has not been built:

- Live GPS capture from a driver device
- Kafka producer publishing to the bus-location topic
- Kafka consumer writing to bus_locations
- End-to-end GPS → Kafka → PostgreSQL pipeline
- Dynamic ETA calculation
- Route subscription
- Notifications
- Superset analytics dashboards

16. Current Status

The table below is the single most concise summary of project status and is intended to be updated every time a feature moves from PLANNED to IN PROGRESS to COMPLETED.

Feature	Status	Notes
GitHub	COMPLETED	Repository live, commits ongoing
Project Structure	COMPLETED	backend/frontend/database/kafka/superset folders in place
PostgreSQL	COMPLETED	Installed, running, connected
Database Schema	COMPLETED	Core tables created; real-time tables designed
FastAPI	COMPLETED	App running via Uvicorn
Database Connection	COMPLETED	SQLAlchemy wired to PostgreSQL

Feature	Status	Notes
Authentication	COMPLETED	bcrypt + JWT working
Authorization	COMPLETED	Role-based checks enforced server-side
Registration	COMPLETED	Public Student registration; Driver/Admin blocked
User Management	COMPLETED	Core user CRUD via API
Student Management	COMPLETED	API implemented
Driver Management	COMPLETED	API implemented
Faculty Management	COMPLETED	API implemented
Bus Management	COMPLETED	API implemented
Route Management	COMPLETED	API implemented
Stop Management	COMPLETED	API implemented
Driver Assignment	COMPLETED	API implemented
Landing Page	COMPLETED	Built and working
Login Page	COMPLETED	Built and working, calls real API
Student Dashboard	COMPLETED	SPA shell with navigation built
Map UI	COMPLETED	Leaflet + OSM rendering; no live data
Driver GPS	PLANNED	Not started
Kafka	PLANNED	Folder reserved; no producer/consumer yet
Real-Time Pipeline	PLANNED	Depends on Driver GPS + Kafka
Dynamic ETA	PLANNED	Not started
Subscriptions	PLANNED	Table designed only
Notifications	PLANNED	Table designed only
Driver Dashboard	PLANNED	Not started
Faculty Dashboard	PLANNED	Not started
Admin Dashboard UI	PLANNED	APIs exist; no UI yet
Superset	PLANNED	Folder reserved; not connected
Deployment	PLANNED	Currently local development only

Table 13: Feature-by-feature project status as of this revision of the document.

17. Development Roadmap

The roadmap is organised into twelve phases. Phases 1–4 are complete; Phases 5–12 remain. Each phase builds on the previous one, so the order below is intentional and is not meant to be parallelised by a two-person team.

Development Roadmap — 12 Phases

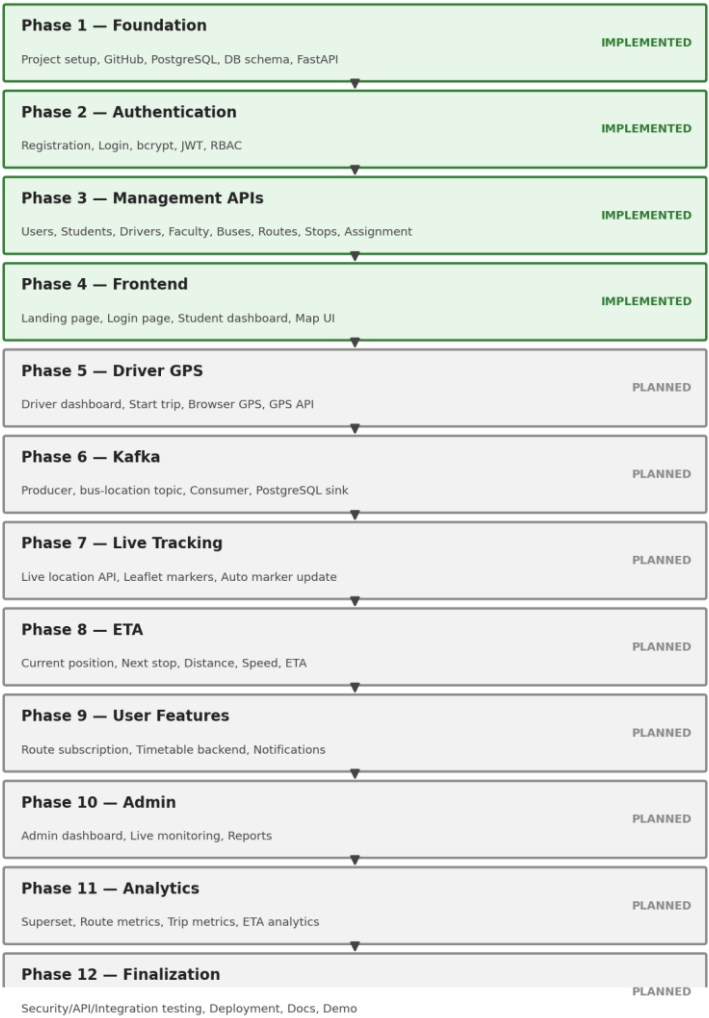


Figure 14: Twelve-phase development roadmap, current progress marked.

Phase	Scope	Status
Phase 1 — Foundation	Project setup, GitHub, PostgreSQL, database schema, FastAPI.	COMPLETED
Phase 2 — Authentication	Registration, Login, bcrypt, JWT, role-based access control.	COMPLETED
Phase 3 — Management APIs	Users, Students, Drivers, Faculty, Buses, Routes, Stops, Driver assignment.	COMPLETED
Phase 4 — Frontend	Landing page, Login page, Student dashboard, Map UI.	COMPLETED
Phase 5 — Driver GPS	Driver dashboard, Start trip, Browser GPS, GPS API.	PLANNED
Phase 6 — Kafka	Producer, bus-location topic, Consumer, write to PostgreSQL.	PLANNED
Phase 7 — Live Tracking	Live location API, Leaflet markers, automatic marker update.	PLANNED
Phase 8 — ETA	Current position, next stop, distance, speed, ETA computation.	PLANNED
Phase 9 — User Features	Route subscription, timetable backend, notifications.	PLANNED

Phase	Scope	Status
Phase 10 — Admin	Admin dashboard UI, live monitoring, reports.	PLANNED
Phase 11 — Analytics	Superset integration, route/trip metrics, ETA analytics.	PLANNED
Phase 12 — Finalization	Security/API/integration testing, deployment, final documentation, PPT, demo.	PLANNED

Table 14: Development roadmap phases and current status.

18. What We Should Do Next

The following ordered list is the immediate, actionable task queue for the team, following directly from Phase 5 onward in the roadmap. It will be trimmed and updated as tasks are completed.

7. Complete/refine the registration UI.
8. Build the student route-subscription backend.
9. Build the driver dashboard.
10. Build driver GPS sharing on the frontend.
11. Create the GPS ingestion API endpoint.
12. Test GPS capture from an Android phone.
13. Integrate a Kafka producer into the GPS endpoint.
14. Create the bus-location Kafka topic.
15. Build a Kafka consumer that writes to PostgreSQL.
16. Store live GPS coordinates in the bus_locations table.
17. Connect live location data to the Leaflet map.
18. Implement ETA calculation.
19. Build the admin dashboard UI.
20. Build the faculty dashboard.
21. Integrate Apache Superset.
22. Carry out final testing (security, API, integration).
23. Deploy the system.

This document will be revised alongside development — each future update should move completed items from this chapter into Chapter 4 (Completed Work) and update Chapters 16 and 17 accordingly.